

MODULE-3

Frontend-CSS and CSS3

Question:1:-What is CSS selector? Provide examples of element, class, and ID selectors.

- A CSS selector is used to select (target) HTML elements so you can style them with CSS.

Selector Type	How It Works	Example	What It Targets
Element Selector	Selects all tags of a type	p { color: blue; }	All <p> tags (paragraphs)
Class Selector	Selects elements with a class name	.box { background: yellow; }	Any element with class="box"
ID Selector	Selects one specific element with an ID	#header { font-size: 24px; }	Element with id="header"

Example:-

Element selector:- <h1 id="main-title">Welcome!</h1>

Class selector:- <p class="info">This is a paragraph.</p>

ID selector:- <p>This is another paragraph.</p>

Question 2: Explain the concept of CSS specificity. How do conflicts between multiple styles get resolved?

- CSS specificity is a set of rules that determines which style gets applied when multiple styles conflict.

- Each CSS selector has a **specificity value**. The **more specific** the rule, the **stronger** it is.

Specificity Ranking (from weakest to strongest):-

Specificity Level	Selector Type	Example	Strength
1. Least Specific	Element selector	p or div	Low
2. Medium	Class selector	.box	Medium
3. Stronger	ID selector	#header	High
4. Most Specific	Inline styles	style="..."	Very High
 Overrides All	!important	color: red !important;	Highest (but avoid using often)

Question 3: What is the difference between internal, external, and inline CSS? Discuss the advantages and disadvantages of each approach.

- CSS means Cascading Style Sheets. It is used to style your webpage — like setting colors, fonts, sizes, etc.
- There are 3 ways to use CSS in your HTML.

- 1. Inline CSS**
- 2. Internal CSS**
- 3. External CSS**

1. Inline CSS

- Inside the HTML tag, using the `style` attribute.

2. Internal CSS

- Inside a `<style>` tag in the `<head>` part of your HTML.

3.External CSS

- In a separate file (like style.css) and linked to HTML using <link>

CSS Box Model

Question 1: Explain the CSS box model and its components (content, padding, border, margin). How does each affect the size of an element?

The CSS Box Model is the way every HTML element (like a <div>, <p>, etc.) is treated like a box. This box has 4 parts:-

1. Content:-

This is where your text, images, or content goes.
You set its **width** and **height**.

2. Padding:-

Space inside the box, around the content.
Pushes the content **away from the edges**.

3. Border:-

A line around the padding and content.
You can set its **thickness, style, and color**.

4. Margin:-

Space outside the box.
Pushes the element away from other elements.

Question 2: What is the difference between border-box and content-box box-sizing inCSS? Which is the default?

Feature	content-box	border-box
What it means	Width/height applies only to content	Width/height includes content + padding + border

Extra space	Padding and border are added outside the content	Padding and border are inside the total size
Total size grows?	Yes — total box gets bigger	No — total size stays the same
Easier layout?	No — you need to do size math	Yes — layout is more predictable
Used by default?	Yes (default value in CSS)	No (you set it manually)
Example	width: 200px means content only	width: 200px includes everything

CSS Flexbox

Question 1: What is CSS Flexbox, and how is it useful for layout design? Explain the terms flex-container and flex-item.

What is CSS Flexbox?

- Flexbox is a layout system in CSS that helps you arrange items in a row or column easily. It stands for "Flexible Box Layout".

1. Flex Container

- This is the **parent box**.
- You make it a flex container using

2. Flex Items

- These are the **children inside the container**.
- You can control their **size, spacing, order, alignment**.

Feature	What it means
Easy Alignment	Center, top, bottom, side-by-side
Responsive Layout	Adjusts for mobile/tablet/desktop
Space Management	Add space between items easily
Flexible Widths	Items grow or shrink to fit space

Reordering Items	You can change the order with CSS
------------------	-----------------------------------

Question 2: Describe the properties justify-content, align-items, and flex-direction used in Flexbox.

flex-direction:-

- **What it does:** Sets the main axis direction (i.e., the direction in which flex items are placed).
- **Values:**
 - row → (default) items go left to right.
 - row-reverse → items go right to left.
 - column → items go top to bottom.
 - column-reverse → items go bottom to top.

justify-content:-

- **What it does:** Aligns items **along the main axis** (which is defined by flex-direction).
- **Values:**
 - flex-start → items go to the start.
 - flex-end → items go to the end.
 - center → items go to the center.
 - space-between → equal space between items.
 - space-around → equal space around each item.
 - space-evenly → equal space between, before, and after items.

align-items:-

- **What it does:** Aligns items **along the cross axis (perpendicular to the main axis)**.

Values:-

- **flex-start** → items align to the top (if row) or left (if column).

- **flex-end** → items align to the bottom (if row) or right (if column).
- **center** → items are centered.
- **stretch** → items stretch to fill the container (default).
- **baseline** → items align by their text baselines.

Example:

Css:-

```
.container {  
  display: flex;  
  flex-direction: row;  
  justify-content: center;  
  align-items: flex-start;  
}
```

CSS Grid

Question 1: Explain CSS Grid and how it differs from Flexbox. When would you use Grid over Flexbox?

CSS Grid is a layout system in CSS that allows you to create two-dimensional layouts — you can arrange items in rows and columns at the same time.

- You define a grid structure using `display: grid`.
- You create **grid tracks** using `grid-template-rows` and `grid-template-columns`.
- It gives you more control over layout **both vertically and horizontally**

What is Flexbox?

Flexbox is also a layout system, but it's **one-dimensional** — you can arrange items in a **row OR column**, not both at once.

- You use display: flex.
- Best for **aligning items in a single line** (either horizontally or vertically).
- It automatically adjusts spacing and size.

Main Differences:-

Feature	Flexbox	Grid
Direction	One-dimensional (row or column)	Two-dimensional (row and column)
Layout Control	Good for content alignment	Good for full page layout
Item Placement	Items flow naturally	You can place items anywhere on the grid
Use Case	Small UI components (buttons, nav)	Entire sections/pages (layout, dashboard)

When to Use Grid:

- When you need a complete page layout.
- When you need precise row and column control.

When to Use Flexbox:

- When laying out items in a line.
- For navbar menus, toolbars, or cards in a row.
- For small components that adapt in size or wrap.

Example of Grid:-

Css:-

```
.container {  
  display: grid;
```

```
grid-template-columns: 1fr 2fr;  
grid-template-rows: auto auto;  
}
```

Example of Flexbox:

Css:-

```
.container {  
  display: flex;  
  flex-direction: row;  
  justify-content: space-between;  
}
```

Question 2: Describe the grid-template-columns, grid-template-rows, and grid-gap properties. Provide examples of how to use them.

grid-template-columns

- Defines the number and size of columns in the grid.
- You can use units like px, %, fr (fraction), or even auto.

Example:-

```
.container {  
  display: grid;  
  grid-template-columns: 1fr 2fr 1fr;  
}
```

This creates 3 columns:

First: 1 fraction

Second: 2 fractions (wider)

Third: 1 fraction

grid-template-rows:-

- What it does: Defines the number and size of rows in the grid.
- Also uses units like px, %, fr, auto.
- **EXAMPLE:-**


```
.container {  
  display: grid;  
  grid-template-rows: 100px auto 50px;  
}
```

This creates 3 rows:

1. First: 100 pixels
2. Second: height based on content (auto)
3. Third: 50 pixels

grid-gap (or use gap)

- What it does: Sets the space between rows and columns.

You can give:

- One value → gap between both rows & columns.
- Two values → row-gap column-gap.

Example:-

```
.container {  
  display: grid;  
  grid-template-columns: 1fr 1fr 1fr;  
  grid-gap: 20px;  
}
```

This creates 3 equal columns with 20px space between all rows and columns.

<style>

```
.container {  
  display: grid;  
  grid-template-columns: 200px 1fr;  
  grid-template-rows: 100px 200px;  
  gap: 10px;  
}  
  
.item {  
  background: lightblue;
```

```
padding: 20px;
text-align: center;
}
</style>

<div class="container">
  <div class="item">1</div>
  <div class="item">2</div>
  <div class="item">3</div>
  <div class="item">4</div>
</div>
```

Responsive Web Design with Media Queries

Question 1: What are media queries in CSS, and why are they important for responsive design?

Media queries are special rules in CSS that allow your website to change its layout or style based on the screen size, device type, or orientation (like portrait or landscape).

Why are Media Queries Important for Responsive Design?

Because they make your website **look good on all devices** — desktops, tablets, and mobile phones.

- They help your content **resize, rearrange, or hide/show** elements.
- Without media queries, a site made for desktops would look broken on phones.
- You write a **@media** rule with a **condition** (like max-width), and inside it, you write the CSS that applies **only when that condition is true**.

BASIC EXAMPLE:-

```
body {  
    background-color: lightblue;  
    font-size: 18px;  
}  
  
/* For screens smaller than 768px */  
@media (max-width: 768px) {  
    body {  
        background-color: lightgreen;  
        font-size: 16px;  
    }  
}
```

Device	CSS Media Query Example
Extra small (phones)	@media (max-width: 600px)
Small (tablets)	@media (max-width: 768px)
Medium (laptops)	@media (max-width: 992px)
Large (desktops)	@media (max-width: 1200px)

Use Cases:

- Change layout from **2-column to 1-column** on mobile.
- Resize **font size**, **image size**, or **buttons**.
- Hide navigation menu and show a **hamburger icon**.

Question 2: Write a basic media query that adjusts the font size of a webpage for screens smaller than 600px.

- On desktops or large screens, the text size is 18px.
- On mobile screens (width 600px or less), the text size becomes 14px for better readability and space adjustment.

CSS Code

```
body {  
    font-size: 18px;  
}  
  
/* For screens smaller than 600px */  
@media (max-width: 600px) {  
    body {  
        font-size: 14px;  
    }  
}
```

Typography and Web Fonts

Question 1: Explain the difference between web-safe fonts and custom web fonts. Why might you use a web-safe font over a custom font?

Web-safe fonts are pre-installed on most operating systems (Windows, macOS, etc.), so you can be almost sure the user's device already has them.

Examples:

- Arial
- Times New Roman
- Verdana
- Georgia
- Courier New
- Tahoma

Why use them:

- No need to download — they load **faster**.
- Always look the **same on all devices**.
- Ideal for **performance-focused** or **email** layouts.
- **Custom Web Fonts**
- **Custom fonts** are not installed on the user's device. You load them from the **web** using services like **Google Fonts**, **Adobe Fonts**, or your own font files.

Examples:

- Roboto
- Open Sans
- Lato
- Montserrat
- Poppins

Why use them:

- Make your site **stand out** with unique style.
- Offer **branding and personality**.
- More control over **typography design**.

Feature	Web-Safe Fonts	Custom Web Fonts
Installed by user?	Yes (pre-installed)	No (loaded via internet)
Load speed	Very fast	Slower (depends on font size/source)
Appearance	Basic but reliable	Stylish and customizable
Internet required?	No	Yes (unless self-hosted)

Why Use Web-Safe Fonts Over Custom Fonts?

You might choose web-safe fonts when:

- You want **fast loading** (especially on slow networks).

- You're building for **older browsers or email clients**.
- You're creating a **simple, clean design**.
- You want to **avoid font loading delays** or fallback issues.

```
body {
  font-family: 'Poppins', Arial, sans-serif;
}
```

Question 2: What is the font-family property in CSS? How do you apply a custom Google Font to a webpage?

1. What is the font-family property in CSS?

The font-family property defines the typeface (font) used for text on a webpage.

You can list multiple fonts:

- If the first font isn't available, the browser tries the next one.
- Always end with a generic family (like serif, sans-serif).

Example:-

```
body {
  font-family: 'Arial', 'Helvetica', sans-serif;
}
```

2. How to apply a custom Google Font to a webpage

To use a Google Font, you need to do two things:

Step 1: Add the Google Fonts <link> in the HTML <head>

Example (for Poppins font):

html

CopyEdit

```
<link      href="https://fonts.googleapis.com/css2?family=Poppins&display=swap"
rel="stylesheet">
```

Step 2: Use font-family in your CSS

css

CopyEdit

```
body {  
  font-family: 'Poppins', sans-serif;  
}
```

Complete Example:

html

CopyEdit

```
<!DOCTYPE html>  
  
<html>  
  
<head>  
  
  <title>Google Font Example</title>  
  
  <link      href="https://fonts.googleapis.com/css2?family=Poppins&display=swap"  
rel="stylesheet">  
  
  <style>  
  
    body {  
  
      font-family: 'Poppins', sans-serif;  
  
      font-size: 18px;  
  
      color: #333;  
  
    }  
  
  </style>  
  
</head>  
  
<body>  
  
  <h1>Hello with Google Font!</h1>  
  
  <p>This paragraph uses the Poppins font.</p>  
  
</body>  
  
</html>
```

